

Rust逆向之循环

1.while循环

如下所示的while循环是最基础的循环结构，在Rust中也只有这种while循环结构。Rust不支持do-while的循环结构，虽然do依然是关键字，但是目前并没有被使用到。

```
fn test_while() {  
    let mut x = 0;  
  
    while x < 10 {  
        x += 1;  
    }  
}
```

对应汇编代码如下，函数会判断x是否小于10，如果大于10则直接退出函数：

```
.text:0000000140001230 test_pro__test_while proc near  
.text:0000000140001230          sub     rsp, 28h  
.text:0000000140001234          mov     dword ptr [rsp+24h], 0 ; x=0  
.text:000000014000123C          ; CODE XREF:  
test_pro__test_while+31↓j  
.text:000000014000123C          cmp     dword ptr [rsp+24h], 10  
.text:0000000140001241          jl     short loc_140001248 ; x<10则跳转  
.text:0000000140001243          add     rsp, 28h  
.text:0000000140001247          retn
```

接下来就是对x进行+1操作，并会判断是否发生整型溢出，如果没有发生，则将x+1的值赋给x，并跳转到上面继续判断x是否小于10。

```

.text:0000000140001248 loc_140001248:                                ; CODE XREF:
test_pro__test_while+11↑j
.text:0000000140001248      mov     eax, [rsp+24h] ; eax=x
.text:000000014000124C      inc     eax           ; eax=x+1
.text:000000014000124E      mov     [rsp+20h], eax ; 保存eax+1
.text:0000000140001252      seto    al
.text:0000000140001255      test    al, 1
.text:0000000140001257      jnz     short loc_140001 ; 发生整型溢出则跳转发生
整型溢出
.text:0000000140001259      mov     eax, [rsp+20h] ; eax=x+1
.text:000000014000125D      mov     [rsp+24h], eax ; x=eax
.text:0000000140001261      jmp     short loc_14000123C

```

2.loop循环

在C/C++中，如果需要使用死循环，一般是用while(true)这种写法。Rust则提供了loop这个关键字，在loop循环中，如果不用break来跳出循环，程序就会一直在这个循环中运行。下面是一个简单的loop循环结构：

```

fn test_loop() {
    let mut x = 0;

    loop {
        if x == 10 {
            break;
        }
        x += 1;
    }
}

```

对应汇编代码如下，同样函数会首先判断x是否不等于10，如果x等于10就会继续向下运行退出函数。

```

.text:0000000140001280 test_pro__test_loop proc near
.text:0000000140001280      sub     rsp, 28h
.text:0000000140001284      mov     dword ptr [rsp+24h], 0 ; x=0
.text:000000014000128C      loc_14000128C:                                ; CODE XREF:
test_pro__test_loop+31↓j
.text:000000014000128C      cmp     dword ptr [rsp+24h], 0Ah
.text:0000000140001291      jnz     short loc_140001298 ; x!=10则跳转
.text:0000000140001293      add     rsp, 28h
.text:0000000140001297      retn

```

如果x不等于10，就会进行x=x+1，并跳转到上面判断x是否等于10的代码：

```
.text:0000000140001298 loc_140001298: ; CODE XREF:
test_pro__test_loop+11↑j
.text:0000000140001298 mov     eax, [rsp+24h] ; eax=x
.text:000000014000129C inc     eax           ; eax=x+1
.text:000000014000129E mov     [rsp+20h], eax ; 保存eax
.text:00000001400012A2 seto    al
.text:00000001400012A5 test    al, 1
.text:00000001400012A7 jnz     short loc_1400012B3 ; 整型溢出则跳转
.text:00000001400012A9 mov     eax, [rsp+20h] ; eax=x+1
.text:00000001400012AD mov     [rsp+24h], eax ; x=eax
.text:00000001400012B1 jmp     short loc_14000128C
```

Rust中的break关键字除了可以跳出循环，还可以指定一个返回值。比如以下代码，当x等于10的时候，break不仅会退出循环，还会返回5+1的值，将其赋给局部变量y。

```
fn test_loop_break() {
    let mut x = 0;

    let y = loop {
        if x == 10 {
            break 5 + 1;
        }
        x += 1;
    };
}
```

对应汇编如下，函数会判断x是否不等于10：

```

.text:000000001400012D0 test_pro__test_loop_break proc near
.text:000000001400012D0
.text:000000001400012D0             sub     rsp, 38h
.text:000000001400012D4             mov     dword ptr [rsp+30h], 0 ; x=0
.text:000000001400012DC             ; CODE XREF:
test_pro__test_loop_break+67↓j
.text:000000001400012DC             cmp     dword ptr [rsp+30h], 10
.text:000000001400012E1             jnz     short loc_1400012F7 ; x!=10则跳转
.text:000000001400012E3             mov     eax, 5
.text:000000001400012E8             inc     eax ; eax=5+1
.text:000000001400012EA             mov     [rsp+2Ch], eax ; 保存eax的值
.text:000000001400012EE             seto    al
.text:000000001400012F1             test    al, 1
.text:000000001400012F3             jnz     short loc_140001317 ; 整型溢出则跳转
.text:000000001400012F5             jmp     short loc_14000130A

```

如果x等于0，则保存5+1的值，并判断是否整型溢出，如果没有溢出就将5+1的值赋给y并退出函数：

```

.text:0000000014000130A loc_14000130A: ; CODE XREF:
test_pro__test_loop_break+25↑j
.text:0000000014000130A             mov     eax, [rsp+2Ch] ; eax=5+1
.text:0000000014000130E             mov     [rsp+34h], eax ; y=eax
.text:00000000140001312             add     rsp, 38h
.text:00000000140001316             retn

```

如果x等于10，则保存x+1的值，并判断是否整型溢出：

```

.text:000000001400012F7 loc_1400012F7: ; CODE XREF:
test_pro__test_loop_break+11↑j
.text:000000001400012F7             mov     eax, [rsp+30h] ; eax=x
.text:000000001400012FB             inc     eax ; eax=x+1
.text:000000001400012FD             mov     [rsp+28h], eax ; 保存eax
.text:00000000140001301             seto    al
.text:00000000140001304             test    al, 1
.text:00000000140001306             jnz     short loc_140001339 ; 整型溢出则跳转
.text:00000000140001308             jmp     short loc_14000132F

```

如果没有溢出，则将x+1的值赋给x，并跳转到上面继续判断x是否等于10的代码：

```

.text:000000014000132F loc_14000132F:                                ; CODE XREF:
test_pro__test_loop_break+38↑j
.text:000000014000132F      mov     eax, [rsp+28h]    ; eax=x+1
.text:0000000140001333      mov     [rsp+30h], eax    ; x=eax
.text:0000000140001337      jmp     short loc_1400012DC

```

3.for循环

Rust中的for循环和C/C++中的for(int i = 0; i < 3; i++)的形式不一样，以下是在Rust中该for循环的例子：

```

fn test_for_iter() {
    let mut x = 0;
    for i in 0..3 {
        x += i;
    }
}

```

在这种情况下，Rust会用下面的结构体来保存for循环中的0和3：

```

pub struct Range<Idx> {
    pub start: Idx,
    pub end: Idx,
}

```

函数一开始就会为start和end赋值，然后调用_ZN63_LTlu20asu20core_iter_traits_collect__IntolteratorGT9into_iter17hb454230bd04b6b64E函数。这个函数执行过后，eax会等于start，edx会等于end。所以执行完这个函数以后，就会将eax和edx赋值给range1变量：

```

.text:0000000140001550 test_pro__test_for_iter proc near      ; CODE XREF:
test_pro__main+13↑p
.text:0000000140001550                                ; DATA XREF:
.pdata:00000001400260F0↓o
.text:0000000140001550
.text:0000000140001550 var_24                = dword ptr -24h
.text:0000000140001550 var_x                = dword ptr -20h
.text:0000000140001550 range                = Range ptr -1Ch
.text:0000000140001550 range1               = Range ptr -14h
.text:0000000140001550 var_ret              = dword ptr -0Ch
.text:0000000140001550 var_8                = dword ptr -8
.text:0000000140001550 var_4                = dword ptr -4
.text:0000000140001550
.text:0000000140001550                sub     rsp, 48h
.text:0000000140001554                mov     [rsp+48h+var_x], 0 ; x=0
.text:000000014000155C                mov     [rsp+48h+range.start], 0 ; 为Range结构体
赋值
.text:0000000140001564                mov     [rsp+48h+range.end], 3
.text:000000014000156C                mov     ecx, [rsp+48h+range.start] ;
core::ops::range::Range<i32>
.text:0000000140001570                mov     edx, [rsp+48h+range.end]
.text:0000000140001574                call
_ZN63_$LT$I$u20$as$u20$core__iter_traits_collect__IntoIterator$GT$9into_iter17hb4542
30bd04b6b64E ;
_$LT$I$u20$as$u20$core..iter_traits.collect..IntoIterator$GT$::into_iter::hb454230bd
04b6b64
.text:0000000140001579                mov     [rsp+48h+range1.start], eax
.text:000000014000157D                mov     [rsp+48h+range1.end], edx

```

接下来会将range1地址作为第一个参数，调用_ZN4core4iter5range101_LTimplu20core_iter_traits_iterator_iteratoru20foru20core_ops_range_RangeLTA\$GT\$GT4next17h2a12ff90d471b83cE函数。这个函数会判断range1.start是否小于range1.end，如果小于，先将range1.start保存到eax，然后将range1.start+=1，在将eax赋值为1。否则的话，就会直接将eax赋值为0，然后退出函数。所以执行完这个函数以后，会首先将edx中保存的range1.start保存到变量i中，在将返回值保存到eax中，在根据返回值决定是否跳转。

```

.text:0000000140001581 loc_140001581:                                ; CODE XREF:
test_pro__test_for_iter+71↓j
.text:0000000140001581                lea     rcx, [rsp+48h+range1] ;
core::ops::range::Range<i32> *
.text:0000000140001586                call
_ZN4core4iter5range101_$LT$impl$u20$core__iter__traits__iterator__Iterator$u20$for$u20$
$core__ops__range__Range$LT$A$GT$$GT$4next17h2a12ff90d471b83cE ;
core::iter::range::_$LT$impl$u20$core..iter..traits..iterator..Iterator$u20$for$u20$co
re..ops..range..Range$LT$A$GT$$GT$::next::h2a12ff90d471b83c
.text:000000014000158B                mov     [rsp+48h+var_i], edx
.text:000000014000158F                mov     [rsp+48h+var_ret], eax
.text:0000000140001593                mov     eax, [rsp+48h+var_ret]
.text:0000000140001597                cmp     rax, 0
.text:000000014000159B                jnz     short loc_1400015A2      ; 返回值不为0则
跳转
.text:000000014000159D                add     rsp, 48h
.text:00000001400015A1                retn

```

如果不跳转，就直接退出函数。如果跳转，就将 $i+x$ 保存到 x 中，然后跳转到上面的代码继续判断：

```

.text:00000001400015A2 loc_1400015A2:                                ; CODE XREF:
test_pro__test_for_iter+4B↑j
.text:00000001400015A2                mov     eax, [rsp+48h+var_i] ; eax=i
.text:00000001400015A6                mov     [rsp+44h], eax ; 保存i
.text:00000001400015AA                add     eax, [rsp+48h+var_x] ; eax=i+x
.text:00000001400015AE                mov     [rsp+24h], eax ; 保存i+x
.text:00000001400015B2                seto    al
.text:00000001400015B5                test    al, 1
.text:00000001400015B7                jnz     short loc_1400015C3 ; 整型溢出则跳转
.text:00000001400015B9                mov     eax, [rsp+24h] ; eax=i+x
.text:00000001400015BD                mov     [rsp+48h+var_x], eax ; x=i+x
.text:00000001400015C1                jmp     short loc_140001581

```